

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №22

Тема: Створення алгоритмів на цикли. Повний перебір

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Ознайомитися з методом повного перебору (brute force), навчитися оцінювати розмір простору перебору та кількість операцій, застосовувати вкладені цикли для перебору комбінацій і оптимізувати перебір за рахунок звуження меж та відсікання зайвих варіантів.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Повний перебір (метод «грубої сили», brute force) — метод розв'язання задачі, при якому послідовно перевіряються **усі** можливі варіанти розв'язку, і з них вибираються ті, що задовольняють умову задачі. Метод простий, гарантовано коректний (якщо перебрано всі варіанти), але часто має велику обчислювальну складність.

2. Схема алгоритму повного перебору:

1. Визначити **простір пошуку** — множину всіх кандидатів (змінні та діапазони їх значень).
2. Перебрати кандидатів циклами (кожній змінній відповідає свій цикл або вкладений цикл).
3. Для кожного кандидата **перевірити умову** задачі.
4. Зберегти підходящі кандидати (підраховувати, запам'ятати перший, найкращий тощо).

3. Розмір простору перебору. Якщо є k змінних і змінна i набуває m_i значень, то кількість кандидатів дорівнює $m_1 \cdot m_2 \cdot \dots \cdot m_k$. Для k змінних із однаковим діапазоном n — n^k . Для перестановок n елементів — $n!$, для підмножин n -елементної множини — 2^n . Саме тому повний перебір придатний лише для невеликих n : так, для $n = 10$ маємо $10! = 3\,628\,800$ перестановок, а для $n = 20$ — вже понад $2 \cdot 10^{18}$.

4. Способи скорочення перебору:

- звуження меж — зменшення діапазонів змінних (наприклад, перевіряти дільники числа n лише до $\sqrt{n}$);
- виключення залежної змінної — замість перебору третьої змінної обчислити її з рівняння ($c = n - a - b$);
- впорядкування перебору ($a < b < c$) — відсікання повторів однакових комбінацій;
- раннє завершення — `break` після першого знайденого розв'язку, якщо потрібен лише він.

5. Інструменти Python для комбінаторного перебору (модуль `itertools`): `product(A, repeat=k)` — декартів добуток; `permutations(A)` — перестановки; `combinations(A, k)`

— комбінації без повторів. Ці функції замінюють вкладені цикли та роблять код компактнішим.

Хід роботи

Завдання 1. Ручний перебір: піфагорові трійки

Знайти вручну всі піфагорові трійки натуральних чисел (a, b, c) , у яких $a < b < c \leq 20$ і $a^2 + b^2 = c^2$.

Порядок виконання:

1. Визначте простір пошуку: кількість трійок (a, b, c) із умовою $a < b < c \leq 20$ (це кількість вибору 3 різних чисел із 20, тобто 1140) і порівняйте з $20^3 = 8000$ трійок без обмежень.
2. Складіть таблицю квадратів чисел від 1 до 20.
3. Для кожного c від 5 до 20 перебирайте a і b ($a < b < c$) і перевіряйте рівність $a^2 + b^2 = c^2$ за таблицею квадратів; результат фіксуйте у таблиці « c — знайдені пари (a, b) ».
4. Випишіть усі знайдені трійки й перевірте, серед яких є «кратні» (наприклад, $(6, 8, 10) = 2 \cdot (3, 4, 5)$); визначте, скільки серед них **примітивних** (числа взаємно прості).
5. Оцініть, як зменшиться кількість перевірок, якщо c не перебирати, а обчислювати як $\sqrt{a^2 + b^2}$ і перевіряти, що результат – ціле число ≤ 20 .

Завдання 2. Ручний перебір: розмін суми монетами

Скількома способами можна набрати суму 10 грн монетами номіналом 1, 2 та 5 грн (порядок монет не важливий)? Виконати повний перебір вручну.

Порядок виконання:

1. Позначте через x_5, x_2, x_1 кількість монет відповідно по 5, 2 та 1 грн; запишіть рівняння $5 \cdot x_5 + 2 \cdot x_2 + x_1 = 10$.
2. Визначте діапазони змінних: $x_5 \in \{0, 1, 2\}$; для кожного x_5 обчисліть залишок і діапазон x_2 (від 0 до залишок // 2).
3. Змінну x_1 не перебирайте, а обчисліть із рівняння: $x_1 = 10 - 5 \cdot x_5 - 2 \cdot x_2$ (це виключення залежної змінної).
4. Складіть таблицю « x_5 — x_2 — x_1 » з усіма способами та порахуйте їхню кількість.
5. Обчисліть кількість перевірок при повному переборі трьох змінних ($x_5 \leq 2, x_2 \leq 5, x_1 \leq 10$) і при переборі двох змінних; зробіть висновок про ефективність виключення залежної змінної.

Завдання 3. Програмна реалізація: прості й досконалі числа, піфагорові трійки

Реалізувати мовою Python повний перебір для кількох задач і порівняти кількість виконаних перевірок при різних варіантах меж перебору.

Порядок виконання:

1. Напишіть функцію `pythagorean(limit)`, що повертає список усіх трійок $a < b < c \leq \text{limit}$, використовуючи три вкладені цикли; перевірте її на `limit = 20` (результат має збігатися з ручним).
2. Напишіть оптимізовану версію `pythagorean_fast(limit)`, у якій `c` обчислюється через `math.isqrt`; переконайтеся, що список трійок такий самий, і порівняйте кількість перевірок для `limit = 100`.
3. Напишіть функцію `is_prime(n)`, що перебирає можливі дільники від 2 до $\sqrt{n}$ включно та повертає `False` при першому знайденому дільнику (`break/return`).
4. Напишіть функцію `perfect_numbers(limit)`, що знаходить усі досконалі числа (сума власних дільників дорівнює самому числу) до `limit = 10000` повним перебором дільників.
5. Напишіть функцію `coin_ways(total, coins)`, що підраховує кількість способів розмінювати суму перебором кількостей двох менших монет і обчисленням залишку; перевірте для `(10, [1, 2, 5]) = 10`.

Приклад реалізації мовою Python:

python

```
import math

def pythagorean(limit):
    found, checks = [], 0
    for a in range(1, limit + 1):
        for b in range(a + 1, limit + 1):
            for c in range(b + 1, limit + 1):
                checks += 1
                if a * a + b * b == c * c:
                    found.append((a, b, c))
    return found, checks

def pythagorean_fast(limit):
    found, checks = [], 0
    for a in range(1, limit + 1):
        for b in range(a + 1, limit + 1):
            checks += 1
            c = math.isqrt(a * a + b * b)
            if c <= limit and c * c == a * a + b * b:
                found.append((a, b, c))
    return found, checks

def is_prime(n):
    if n < 2:
        return False
    d = 2
    while d * d <= n:
        if n % d == 0:
            return False
        d += 1
    return True

def perfect_numbers(limit):
    result = []
    for n in range(2, limit + 1):
        s = 1
        for d in range(2, n // 2 + 1):
```

```

        if n % d == 0:
            s += d
        if s == n:
            result.append(n)
    return result

def coin_ways(total, coins):
    c1, c2, c3 = coins
    count = 0
    for x3 in range(total // c3 + 1):
        for x2 in range((total - c3 * x3) // c2 + 1):
            rest = total - c3 * x3 - c2 * x2
            if rest % c1 == 0:
                count += 1
    return count

t1, n1 = pythagorean(20)
t2, n2 = pythagorean_fast(20)
print("Трійки при limit = 20:", t1)
print("Збігаються з оптимізованою версією:", t1 == t2, "| перевірок:", n1, "проти", n2)
_, n1 = pythagorean(100)
t3, n3 = pythagorean_fast(100)
print(f"limit = 100: знайдено трійок: {len(t3)}; перевірок {n1} проти {n3}")
primes = [n for n in range(2, 60) if is_prime(n)]
print("Прості числа < 60:", primes)
print("Кількість простих до 10000:", sum(is_prime(n) for n in range(10001)))
print("Досконалі числа до 10000:", perfect_numbers(10000))
print("Способів розмінати 10 грн (1, 2, 5):", coin_ways(10, (1, 2, 5)))
print("Способів розмінати 100 грн (1, 2, 5):", coin_ways(100, (1, 2, 5)))

```

Результат виконання програми:

```

Трійки при limit = 20: [(3, 4, 5), (5, 12, 13), (6, 8, 10), (8, 15, 17), (9, 12, 15),
(12, 16, 20)]
Збігаються з оптимізованою версією: True | перевірок: 1140 проти 190
limit = 100: знайдено трійок: 52; перевірок 161700 проти 4950
Прості числа < 60: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59]
Кількість простих до 10000: 1229
Досконалі числа до 10000: [6, 28, 496, 8128]
Способів розмінати 10 грн (1, 2, 5): 10
Способів розмінати 100 грн (1, 2, 5): 541

```

Висновок за результатами: повний перебір знаходить шість піфагорових трійок зі сторонами до 20 — так само, як і ручний перебір: (3, 4, 5), (5, 12, 13), (6, 8, 10), (8, 15, 17), (9, 12, 15), (12, 16, 20). Обчислення сторони с замість її перебору скорочує кількість перевірок для limit = 100 із 161700 до 4950 (приблизно у 33 разів) за збереження того самого результату. Обмеження перебору дільників значенням $\sqrt{n}$ дозволяє швидко знайти всі 1229 простих чисел до 10000. Досконалі числа до 10000 – 6, 28, 496 та 8128; кількість способів розмінати 10 грн дорівнює 10, як і в ручному розрахунку.

Завдання 4. Дослідження складності повного перебору

Дослідити, як межі перебору впливають на кількість операцій, розв'язати задачу про «щасливі квитки» та оцінити час перебору паролів.

Порядок виконання:

1. Порівняйте кількість ділень для підрахунку простих чисел до $N = 10000$ за трьома схемами: дільники від 2 до $n-1$; від 2 до $n // 2$; від 2 до $\sqrt{n}$ –

у кожному варіанті із `break` після першого дільника. Занесіть кількість ділень у таблицю «схема — кількість операцій».

2. Розв'яжіть задачу про «щасливі квитки»: скільки існує шестицифрових номерів від 000000 до 999999, у яких сума перших трьох цифр дорівнює сумі останніх трьох? Виконайте повний перебір усіх 10^6 номерів.
3. Розв'яжіть цю ж задачу швидше: обчисліть кількість трицифрових комбінацій із сумою s для кожного s від 0 до 27 і підсумуйте квадрати; перевірте збіг відповідей.
4. Складіть таблицю часу повного перебору пароля довжиною 4, 6, 8, 10 символів для алфавіту з 10 цифр та з 62 символів (цифри + малі та великі латинські літери) за швидкості 10^9 перевірок за секунду.
5. Сформулюйте висновок про граничні розміри задач, які можна розв'язати повним перебором.

Приклад реалізації мовою Python:

python

```
import math

def primes_count(limit, scheme):
    ops, count = 0, 0
    for n in range(2, limit + 1):
        if scheme == "n-1":
            top = n - 1
        elif scheme == "n/2":
            top = n // 2
        else:
            top = math.isqrt(n)
        prime = True
        for d in range(2, top + 1):
            ops += 1
            if n % d == 0:
                prime = False
                break
        count += prime
    return count, ops

for scheme in ("n-1", "n/2", "sqrt"):
    cnt, ops = primes_count(10000, scheme)
    print(f"схема {scheme:>4}: простих {cnt}, ділень {ops}")

lucky = 0
for n in range(1000000):
    s = f"{n:06d}"
    if int(s[0]) + int(s[1]) + int(s[2]) == int(s[3]) + int(s[4]) + int(s[5]):
        lucky += 1
ways = [0] * 28
for a in range(10):
    for b in range(10):
        for c in range(10):
            ways[a + b + c] += 1
print("Щасливих квитків (повний перебір):", lucky, "; формулою:", sum(w * w for w in ways))

print("\nЧас перебору паролів (10^9 перевірок/с):")
for length in (4, 6, 8, 10):
    row = []
    for alphabet in (10, 62):
        seconds = alphabet ** length / 1e9
        row.append(f"{seconds:.3g} c" if seconds < 3600 else f"{seconds / 86400:.3g} діб")
    print(f" довжина {length:>2}: 10 цифр -> {row[0]:>10}; 62 символи -> {row[1]:>12}")
```

Результат виконання програми:

```
схема n-1: простих 1229, ділень 5775223
схема n/2: простих 1229, ділень 2907640
схема sqrt: простих 1229, ділень 117527
Щасливих квитків (повний перебір): 55252 ; формулою: 55252
```

Час перебору паролів (10^9 перевірок/с):

довжина 4:	10 цифр ->	1e-05 с;	62 символи ->	0.0148 с
довжина 6:	10 цифр ->	0.001 с;	62 символи ->	56.8 с
довжина 8:	10 цифр ->	0.1 с;	62 символи ->	2.53 діб
довжина 10:	10 цифр ->	10 с;	62 символи ->	9.71e+03 діб

*Висновок за результатами: усі три схеми знаходять однакову кількість простих чисел (1229), але звуження меж перебору зменшує кількість ділень з **5775223** до **117527** — приблизно у 49 раз. Повний перебір 10^6 номерів квитків дає **55252** щасливих квитків, що збігається з відповіддю за комбінаторною формулою, яка обходить лише 1000 комбінацій. Кількість паролів зростає експоненційно ($alphabet^{length}$): 4-цифровий PIN перебирається миттєво, а пароль із 10 символів із 62-символьного алфавіту вимагає близько 9700 діб (понад 26 років) перебору навіть за мільярд перевірок за секунду — тому повний перебір придатний лише для задач обмеженого розміру, а для більших потрібні розумніші алгоритми.*

Контрольні питання

1. У чому суть методу повного перебору? Які його переваги та недоліки?
2. Як оцінити розмір простору перебору для k змінних? Скільки існує перестановок n елементів і підмножин n -елементної множини?
3. Які способи скорочення перебору ви знаєте? Чому достатньо перебирати дільники числа до $\sqrt{n}$?
4. Що таке виключення залежної змінної? Поясніть на прикладі піфагорових трійок або розміну монет.
5. Навіщо в умові перебору піфагорових трійок вимагають $a < b < c$?
6. Що таке досконале число? Наведіть перші чотири досконалі числа.
7. Для яких розмірів задач повний перебір застосовний на практиці, а для яких — ні?
8. Які функції модуля `itertools` замінюють вкладені цикли при переборі комбінацій?

Зміст звіту

1. Тема, мета роботи.
2. Ручні розрахунки та таблиці переборів (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Таблиці порівняння кількості операцій і часу перебору.
5. Висновки.